

# Machine Learning

Friday seminar

---

Hansol Park

April 5, 2019

1. Introduction
2. The perceptron algorithm
3. Kernel Functions
4. Generalizing to New Data
5. Overfitting and Uniform Convergence
6. Illustrative Examples and Occam's Razor
  - Learning Disjunctions

## 5.1 Introduction

---

## 5.1 Introduction

Machine learning algorithms are general purpose tools for generalizing from data.

Application: Speech recognition, Document classification, Automated driving, Computational science, and Decision support.

## 5.1 Introduction: The core problem

### The core problem

A core problem is learning a good classification rule from labeled data.

Domain of interest ( $\mathcal{X}$ ) called the instance space, such as the set of email messages or patient records, and a classification task.

Usually we set  $\chi = \{0, 1\}^d$  or  $\chi = \mathbb{R}^d$ .

For example, we can classify following things.

- Spam mail or Non-spam mail  
⇒ Classified into two categories(positive class, negative class)
- Proper medical treatment  
⇒ Multi-way classification

## 5.1 Introduction: The core problem

### Given data

Set  $S$  of labeled training examples.



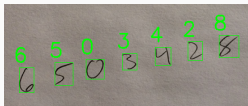
### Result

Generalized method to classification.

## 5.1 Introduction: The core problem(Example)



Set  $S$  of labeled training examples.



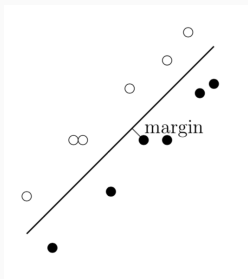
Generalized method to classification.

## 5.1 Introduction: How to learn.

### Example: Classifying email messages

We might find a set of highly indicative words such that every spam email in the training data has at least one of these words and none of the non-spam emails has any of them. For examples, [AD] or 광고.

⇒ This simple algorithm is quite dangerous, because "Every E-mail containing [AD] or 광고" will be classified to "Spam mails".



Margin of linear separator

## 5.2 The perceptron algorithm

---

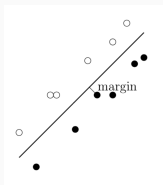
## 5.2 The perceptron algorithm

### Purpose of the Perceptron Algorithm

For the problem of assigning positive and negative weights to features.

In the E-mail classification,

- Spam E-mails have positive weights,
- Non-spam E-mails have negative weights.



We want to find this margin between "Spam E-mails" and "Non-spam E-mails."

## 5.2 The perceptron algorithm

### Purpose of the Perceptron Algorithms in mathematical notation

Let the set  $S \subset \mathbb{R}^n$  is given. Then there is a weights of  $s_i = +1$  or  $-1$  for every vector  $x_i$  in  $S$ . We want to find the vector  $w^*$  such that

1.  $x_i^T w^* > 0$  for all  $i$  with  $s_i = +1$ ,
2.  $x_i^T w^* < 0$  for all  $i$  with  $s_i = -1$ .

Since  $w^*$  with satisfying above condition for all  $i \in I$  cannot be exist, we want to find the most efficient  $w^*$ .

$\Rightarrow$  In the perceptron algorithm, we assume that such vector  $w^*$  exists.

## 5.2 The perceptron algorithm

### The perceptron algorithm

1. Start with the all-zeroes weight vector  $w = 0$ .
2. Let  $x \in S$  be an example for which  $x^T w$  does not have the correct sign.
3. Update as follows:
  - (a) If  $x$  is a positive example, let  $w \leftarrow w + x$ .
  - (b) If  $x$  is a negative example, let  $w \leftarrow w - x$ .

### Remark

- If  $x$  is a positive example,

$$x^T w \rightarrow x^T (w + x) = x^T w + x^T x \geq x^T w \Rightarrow \text{So } x^T w \text{ increases!}$$

- If  $x$  is a negative example,

$$x^T w \rightarrow x^T (w - x) = x^T w - x^T x \leq x^T w \Rightarrow \text{So } x^T w \text{ decreases!}$$

- Now we are concerned that this algorithm works properly, but following theorem will give us the answer.

## 5.2 The perceptron algorithm

### Theorem 5.1

If there exists a vector  $w^*$  such that  $x^T w^* \geq 1$  for all positive examples  $x \in S$  and  $x^T w^* \leq -1$  for all negative examples  $x \in S$  (i.e., a linear separator of margin  $\gamma = 1/|w^*|$ ), then the number of updates made by the Perceptron algorithm is at most  $R^2 |w^*|^2$ , where  $R = \max_{x \in S} |x|$ .

## 5.2 The perceptron algorithm

**Proof.** Let's fix the  $w^*$  such that "If" condition of the theorem.

1. We'll keep track of two quantities,  $w^T w^*$  and  $|w|^2 (= w^T w)$
2. Each time we make an update,  $w^T w^*$  increases by at least 1.

- If  $x$  is a positive example, then

$$w^T w^* \rightarrow (w + x)^T w^* = w^T w^* + x^T w^* \geq w^T w^* + 1$$

- If  $x$  is a negative example, then

$$w^T w^* \rightarrow (w - x)^T w^* = w^T w^* - x^T w^* \geq w^T w^* + 1$$

3. On each update, we claim that  $|w|^2$  increases by at most  $R^2$ .

- Positive examples  $x$  when  $x^T w < 0$

$$(x + w)^T (x + w) = |w|^2 + 2x^T w + |x|^2 \leq |w|^2 + |x|^2 \leq |w|^2 + R^2$$

- Negative examples  $x$  when  $x^T w > 0$

$$(x - w)^T (x - w) = |w|^2 - 2x^T w + |x|^2 \leq |w|^2 + |x|^2 \leq |w|^2 + R^2$$

4. If we make  $M$  updates, then  $w^T w^* \geq M$ ,  $|w|^2 \leq MR^2$ .

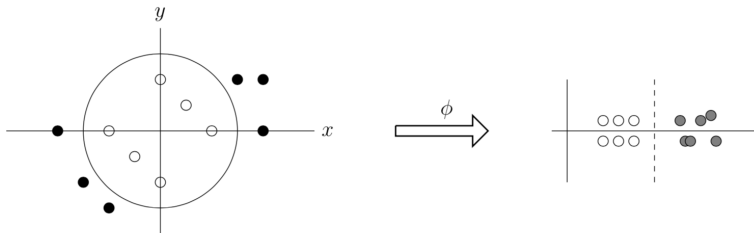
$\Rightarrow$  From Cauchy-Schwarz inequality, we can obtain  $M \leq R^2 |w^*|^2$ .  $\square$

## 5.3 Kernel Functions

---

## 5.3 Kernel Functions

- In this section, we consider a related issue.
- Suppose the positive and negative examples are not linearly separable.

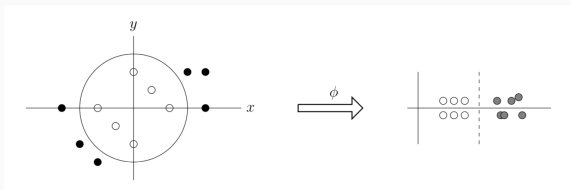


## 5.3 Kernel Functions

### How to choose Kernel function

- Let there are two groups of data  $A$  and  $B$ . We want to find  $\phi(a) = \alpha$  for all  $a \in A$  and  $\phi(b) = \beta$  for all  $b \in B$ .
- We want to make the kernel function  $K(x, x') = \phi(x)^T \phi(x')$  for some vector valued function  $\phi$ .
- We can consider  $K(x, x')$  is an inner product of modified vectors  $\phi(x)$  and  $\phi(x')$ . (Definition of legal kernel)
- Since  $K(x, x) = \phi(x)^T \phi(x)$  is a scalar, we can easily define  $K(x, x)$  more than vectors  $\phi(x)$ .
- Now we can apply perceptron algorithm to  $\phi(x)$ .

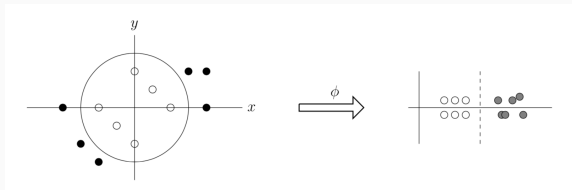
## 5.3 Kernel Functions: Example of Kernel Functions



- Let's consider that data is given as left figure.
- Now we want to find the proper map  $\phi$ .
- Since we want to classify the data with distance to origin, we can set the kernel function as following:

$$K(x, x') = (1 + x^T x')^k \text{ for some } k.$$

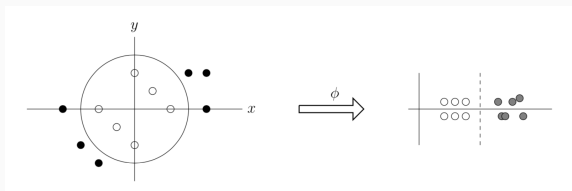
## 5.3 Kernel Functions: Example of Kernel Functions



- If we put  $d = 2, k = 2$ , where  $d$  is a dimension of  $x$ , we can obtain following:

$$\begin{aligned} K(x, x') &= (1 + x_1 x'_1 + x_2 x'_2)^2 \\ &= 1 + 2x_1 x'_1 + 2x_2 x'_2 + x_1^2 x_1'^2 + 2x_1 x_2 x'_1 x'_2 + x_2^2 x_2'^2 \\ &= (1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, \sqrt{2}x_1 x_2, x_2^2)^T (1, \sqrt{2}x'_1, \sqrt{2}x'_2, x_1'^2, \sqrt{2}x'_1 x'_2, x_2'^2) \\ &= \phi(x)^T \phi(x'). \end{aligned}$$

## 5.3 Kernel Functions: Example of Kernel Functions



- $\phi(x) = (1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, \sqrt{2}x_1x_2, x_2^2)$
- Let given circle in left figure be  $x_1^2 + x_2^2 = 4$ . Then we can obtain

$$w^* = (-4, 0, 0, 1, 0, 1).$$

## 5.3 Kernel Functions: Properties of Kernels

### Recall: Definition of Legal Kernels

$K(x, x')$  is a legal kernel if  $K(x, x') = \phi(x)^T \phi(x')$  for some functions  $\phi$ .

### Properties of Legal Kernels(Theorem 5.2)

Suppose  $K_1$  and  $L_2$  are kernel functions. Then

1. For any constant  $c \geq 0$ ,  $cK_1$  is a legal kernel. In fact, for any scalar function  $f$ , the function  $K_3(x, x') = f(x)f(x')K_1(x, x')$  is a legal kernel.
2. The sum  $K_1 + K_2$ , is a legal kernel.
3. The product  $K_1K_2$ , is a legal kernel.

**Proof** First, let  $K_1(x, x') = \phi_1(x)^T \phi_1(x')$  and  $K_2(x, x') = \phi_2(x)^T \phi_2(x')$ .

## 5.3 Kernel Functions: Properties of Kernels(Not in the book)

1.  $cK_1(x, x') = (\sqrt{c}\phi_1(x))^T(\sqrt{c}\phi_1(x'))$   
 $K_3(x, x') = f(x)f(x')K_1(x, x') = (f(x)\phi_1(x))^T(f(x')\phi_1(x'))$
2. Let  $\phi(x) = [\phi_1(x), \phi_2(x)]$ . Then we can obtain following:

$$\begin{aligned}(K_1 + K_2)(x, x') &= \phi_1(x)^T \phi_1(x') + \phi_2(x)^T \phi_2(x') \\ &= \phi(x)^T \phi(x').\end{aligned}$$

3. Let  $\Phi^{i,j}(x) = \phi_1^i(x)\phi_2^j(x)$ , then we can obtain following:

$$\begin{aligned}K_1(x, x')K_2(x, x') &= \phi_1(x)^T \phi_1(x')\phi_2(x)^T \phi_2(x') \\ &= \sum_{i,j} \phi_1^i(x)\phi_1^i(x')\phi_2^j(x)\phi_2^j(x') \\ &= \sum_{i,j} (\phi_1^i(x)\phi_2^j(x))(\phi_1^i(x')\phi_2^j(x')) \\ &= \Phi(x)^T \Phi(x')\end{aligned}$$

□

## 5.4 Generalizing to New Data

---

## 5.4 Generalizing to New Data

To formalize the learning problem, assume there is some probability distribution  $D$  over the instance space  $\mathcal{X}$  such that

1. our training set  $S$  consists of points drawn independently at random from  $D$ .
  2. our objective is to predict well on new points that are also drawn from  $D$ .
- $c^*$ : Target concept, denote the subset of  $\mathcal{X}$  corresponding to the positive class for the binary classification we are aiming to make
  - $h$  : Hypothesis, which is close to  $c^*$  with respect to distribution  $D$ .

## 5.4 Generalizing to New Data

Symmetry difference between two sets is defined as following:

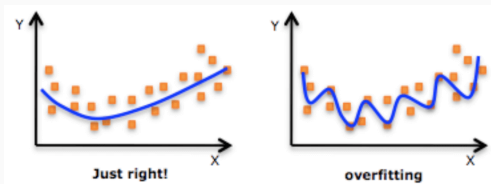
$$A \Delta B = (A - B) \cup (B - A).$$

- The true error of  $h$  is  $err_D(h) := Prob(h \Delta c^*)$ .
- The training error of  $h$ , denoted  $err_S(h) := |S \cap (h \Delta c^*)|/|S|$ .  
Note that  $S$  is assumed to consist of points randomly drawn from  $D$ .

## 5.4 Generalizing to New Data

### Overfitting

Overfitting is a concern because algorithms will typically be optimizing over the training sample.



⇒ To analyze overfitting, we introduce the notion of a **hypothesis class**.

## 5.4 Generalizing to New Data: Hypothesis class

### Definition of Hypothesis class

An hypothesis class  $\mathcal{H}$  over  $\mathcal{X}$  is a collection of subsets of  $\mathcal{X}$ , called hypotheses.

For instance,

- The class of *intervals* over  $\mathcal{X} = \mathbb{R}$  is

$$\mathcal{H} = \{[a, b] | a \leq b\}.$$

- The class of linear separators over  $\mathcal{X} = \mathbb{R}^n$  is

$$\mathcal{H} = \{\{x \in \mathbb{R}^n | w^T x \geq \alpha\} | w \in \mathbb{R}^d, \alpha \in \mathbb{R}\}.$$

## 5.4 Generalizing to New Data

Before giving our first result of this form, we note that it will often be convenient to associate each hypotheses with its  $\{-1, 1\}$ -valued indicator function

$$h(x) = \begin{cases} +1 & (x \in h) \\ -1 & (x \notin h) \end{cases}$$

In this case

$err_D(h) = Prob_{x \sim D}[h(x) \neq c^*(x)]$ : True error

$err_S(h) = Prob_{x \sim S}[h(x) \neq c^*(x)]$ : Training error.

## 5.5 Overfitting and Uniform Convergence

---

## 5.5 Overfitting and Uniform Convergence

To keep things simple, we assume our hypothesis class  $\mathcal{H}$  is finite.

⇒ Later, we will see how to extend these results to infinite classes as well.

I want to introduce three theorems related with uniform convergence.

## 5.5 Overfitting and Uniform Convergence

### Theorem 5.3

Let  $\mathcal{H}$  be an hypothesis class and let  $\epsilon$  and  $\delta$  be greater than zero. If a training set  $S$  of size

$$n \geq \frac{1}{\epsilon} \left( \ln |\mathcal{H}| + \ln \left( \frac{1}{\delta} \right) \right)$$

is drawn from distribution  $D$ , then with probability greater than or equal to  $1 - \delta$  every  $h$  in  $\mathcal{H}$  with true error  $err_D(h) \geq \epsilon$  has training error  $err_S(h) > 0$ . Equivalently, with probability greater than or equal to  $1 - \delta$ , every  $h \in \mathcal{H}$  with training error zero has true error less than  $\epsilon$ .

[I'll omit the proof.]

### Remark

We can know that, there is a hypothesis  $h$  in  $\mathcal{H}$  with zero training error. (Not in mathematical sense.)

## 5.5 Overfitting and Uniform Convergence

### Theorem 5.4(Hoeffding bounds)

Let  $x_1, x_2, \dots, x_n$  be independent  $\{0, 1\}$ -valued random variables with  $\text{Prob}(x_i = 1) = p$ . Let  $s = \sum_i x_i$ . For any  $0 \leq \alpha \leq 1$ ,

$$\text{Prob}(x/n > p + \alpha) \leq e^{-2n\alpha^2},$$

$$\text{Prob}(x/n < p - \alpha) \leq e^{-2n\alpha^2}.$$

[I'll omit the proof.]

**Remark** We can also obtain above result from the law of large numbers.

## 5.5 Overfitting and Uniform Convergence

### Theorem 5.5(Uniform convergence)

Let  $\mathcal{H}$  be a hypothesis class and let  $\epsilon$  and  $\delta$  be greater than zero. If a training set  $S$  of size

$$n \geq \frac{1}{2\epsilon^2} \left( \ln |\mathcal{H}| + \ln \left( \frac{2}{\delta} \right) \right),$$

is drawn from distribution  $D$ , then with probability greater than or equal to  $1 - \delta$ , every  $h$  in  $\mathcal{H}$  satisfies  $|\text{err}_S(h) - \text{err}_D(h)| \leq \epsilon$ .

[I'll omit the proof.]

**Remark** If the training set  $S$  is sufficiently big, we can train it well.

## 5.6 Illustrative Examples and Occam's Razor

---

## 5.6.1 Learning Disjunctions

---

## 5.6.1 Learning Disjunctions

- Consider the instance space  $\mathcal{X} = \{0, 1\}^d$
- Disjunction (an OR;  $\vee$ ) is given as following:

$$x \vee y = \begin{cases} 1 & (x \text{ or } y \text{ is } 1) \\ 0 & (\text{Otherwise}) \end{cases}$$

- If  $S$  has  $d$  elements, the size of  $\mathcal{H}$  is  $2^d$ .  
 $\Rightarrow$  We can choose  $2^d$  subset of  $S$ . ( $\mathcal{H}$  is containing  $x_{i_1} \vee x_{i_2} \vee \dots \vee x_{i_k}$ .)
- We want to find disjunction as hypothesis in  $\mathcal{H}$  efficiently.  
 $\Rightarrow$  Simple Disjunction Learner

## 5.6.1 Learning Disjunctions

### Lemma 5.6

The Simple Disjunction Learner produces a disjunction  $h$  that is consistent with the sample  $S$  (i.e., with  $err_S(h) = 0$ ) whenever the target concept is indeed a disjunction.

This lemma tell us  $\Rightarrow$  We can always find  $h$  if  $h$  exists.

**THANK YOU FOR LISTENING!**